Documentação Técnica de Arquitetura — Sistema de Despacho de Viaturas (SDV)

1. Relevância do Problema, Modelo de Negócio e Proposta de Valor

Problema e Relevância

O domínio atua no setor crítico de segurança pública, onde a alocação de recursos físicos (viaturas) é escassa e disputada. O problema central é garantir que múltiplas centrais de operações possam despachar viaturas de forma simultânea sem que ocorra duplicidade (duas centrais alocando a mesma viatura) ou perda de chamados.

Modelo de Negócio e Proposta de Valor

A solução entrega valor direto para a operação ao garantir confiabilidade absoluta e tempo de resposta em tempo real.

- **Para o Operador:** A certeza de que a viatura mais próxima será despachada, com atualização de estado imediata na interface, sem necessidade de recarregar a tela.
- **Para a Instituição:** Rastreabilidade completa e auditabilidade de ponta a ponta (ingestão durável de eventos) e alta disponibilidade, mesmo sob falhas parciais dos sistemas.

2. Modelagem dos Nós, Separação de Responsabilidades e Protocolos

Arquitetura Geral

O sistema adota o padrão de microsserviços orientados ao domínio (Database-per-service).

Nós e Responsabilidades

- **Serviço de Ocorrências:** Gerencia o ciclo de vida dos chamados, ingestão inicial e dados das centrais de operação.
- **Serviço de Viaturas:** Detém o controle absoluto sobre a malha de recursos físicos (frota) e a máquina de estados das viaturas.
- **API Gateway:** Ponto único de entrada atuando como orquestrador de requisições.

Protocolos de Comunicação

- **Síncrona (Externa/Interna):** GraphQL Federation (Apollo) via HTTP/REST. O cliente consome um Supergraph unificado, enquanto o Gateway compõe as requisições para os subgrafos subjacentes.
- **Assíncrona (Interna):** Redis Streams para ingestão de eventos desacoplada (garantindo durabilidade) e Redis Pub/Sub para propagação de eventos em tempo real (GraphQL Subscriptions para WebSockets).

3. Recursos Computacionais, Bancos de Dados e Balanceamento

Infraestrutura e Dados

- **Bancos de Dados:** PostgreSQL isolado por serviço (ocorrencias_db e viaturas_db). A relação entre tabelas de domínios distintos ocorre exclusivamente por referências lógicas (UUIDs), eliminando chaves estrangeiras físicas transversais.
- **Armazenamento em Memória/Mensageria:** Redis atua simultaneamente como gerenciador de locks distribuídos, cache de posições geográficas (estado quente da frota) e broker de mensageria.
- **Balanceamento de Carga:** Load balancers distribuem o tráfego HTTP de entrada entre múltiplas réplicas dos serviços. Como as aplicações (NestJS) são estritamente stateless, não há necessidade de afinidade de sessão (sticky sessions).

4. Segurança e Gestão de Acessos

Proteção do Sistema e de Borda

- **Autenticação e Autorização:** Implementação de JWT (JSON Web Tokens) com assinaturas HS256/RS256. A autorização baseia-se em claims no token para garantir o princípio do privilégio mínimo (ex: uma central só manipula suas próprias ocorrências).
- **Proteção de Borda (API Gateway):** Rate Limiting por IP ou Central para mitigar ataques de negação de serviço. Análise de profundidade e complexidade de queries GraphQL (graphql-depth-limit) para prevenir sobrecarga intencional do banco de dados por requisições aninhadas. Headers de segurança injetados via Helmet e políticas restritas de CORS.
- **Gestão de Segredos:** Credenciais (Tokens, Connection Strings) nunca são inseridas no código. Utiliza-se injeção via variáveis de ambiente integradas a cofres como o GitHub Secrets, segmentadas por ambiente (Dev, Staging, Prod).

5. Elasticidade e Auto Scaling

Estratégias de Escalonamento Horizontal

- **Políticas de Auto Scaling:** Mecanismos monitoram a utilização de recursos. Um consumo sustentado de 80% de CPU ou Memória por um período de 2 a 5 minutos atua como gatilho de scale-out (adição de réplicas).
- **Scale-in e Drenagem:** Quando a carga diminui (abaixo de 30%), o sistema aplica a remoção de réplicas utilizando connection draining (drenagem de conexões) para garantir que chamadas em andamento não sejam cortadas abruptamente.
- **Proteção do Banco de Dados:** Para suportar a elasticidade agressiva dos microsserviços sem exaurir portas no PostgreSQL, implementa-se Connection Pooling (PgBouncer ou nativo do Prisma).

6. Resiliência e Tolerância a Falhas

Mecanismos de Recuperação e Estabilidade

- **Circuit Breaker:** Protege chamadas inter-serviços. Se o Serviço de Viaturas apresentar latência crítica ou falha, o circuito se abre (padrão fail-fast), impedindo o colapso em cascata do Serviço de Ocorrências.
- **Retries com Backoff Exponencial:** Utilizados estritamente para falhas de rede transitórias, mitigando picos momentâneos de indisponibilidade.
- **Filas Duráveis e Degradação Graciosa:** Ocorrências recebidas durante falhas no despacho não são descartadas. Elas ficam retidas em um Redis Stream (com Consumer Groups). Quando o sistema normaliza, o processamento é retomado de forma assíncrona.
- **Idempotência:** O consumo de mensagens das filas garante que o reprocessamento causado por um crash de sistema não crie despachos ou ocorrências duplicadas, baseando-se em IDs transacionais.

7. Resolução de Nomes e Roteamento de Tráfego

DNS e Mapeamento

- **Mapeamento Externo:** Os domínios são versionados e segregados por ambiente diretamente no subdomínio (ex.: api.sdv.exemplo.com para produção, staging.api.sdv.exemplo.com para homologação).
- **Resolução Interna:** O roteamento leste-oeste (entre microsserviços) não depende de IPs estáticos. Utiliza-se a resolução de DNS do orquestrador de contêineres. O Gateway encaminha requisições baseadas no nome do serviço na rede interna (ex.: http://ocorrencias-service:4001).